

LLM SCA How far are we?

Abstract

1 Motivation

The proliferation of Large Language Models (LLMs) has been dramatically accelerated by collaborative platforms like the [Hugging Face Hub](#), which serves as a central repository for model sharing. A cornerstone of this ecosystem is the iterative development of new models by building upon existing ones through techniques such as fine-tuning, merging, and quantization. This process creates a complex web of inter-model dependencies, which we define as **model genealogy** or **lineage**. A clear understanding of a model's genealogy is paramount for ensuring traceability, reproducibility, and compliance, as derivative models often inherit biases, vulnerabilities, or licensing constraints from their predecessors.

However, the mechanisms currently available on the Hugging Face Hub for documenting this critical lineage information are frequently inconsistent, incomplete, or entirely absent. This lack of a systematic and enforceable standard for provenance documentation poses a significant barrier to researchers and developers. The most visible symptom of this issue is the difficulty, or outright impossibility, of programmatically constructing a model's lineage—a structure ideally represented as a “Model Tree”. This is not a superficial interface problem but a direct consequence of fundamental flaws in how provenance data is recorded at its source.

To illustrate the severity of this challenge, we will dissect two distinct modes of documentation failure, each representing a progressively worsening degree of information obfuscation.

1.1 Problem 1: Lineage Concealed in Unstructured Text

In the most common scenario, lineage information exists but is locked within free-form text, rendering it inaccessible to automated tools. This forces researchers into a manual, error-prone process of discovery.

Case Study 1: DavidAU/NSFW_DPO_Noromaid-7b-GGUF

The Hugging Face repository of this model does not include a model tree; however, its model card explicitly states:

NSFW_DPO_Noromaid-7b-Mistral-7B-Instruct-v0.1 is a merge of the following models: mistralai/Mistral-7B-Instruct-v0.1 and athirpath/NSFW_DPO_Noromaid-7b.

Notably, the Hugging Face pages of both referenced models in the model card contain their respective model trees. This reliance on unstructured documentation makes the automated construction of a lineage graph infeasible. It imposes a significant manual burden for identifying even a direct parent and completely obstructs any large-scale, systematic analysis of the model ecosystem.

Case Study 2: timdettmers/guanaco-13b

Another example is the timdettmers/guanaco-13b model. While this model is a fine-tuned version of LLaMA, this pivotal relationship is not captured in any structured, machine-readable metadata. To discover this link, a user must manually navigate to the model's repository, open the “Model Card” (a Markdown file), and parse through natural language prose to find the statement:

Guanaco models are open-source finetuned chatbots obtained through 4-bit QLoRA tuning of LLaMA base models...

1.2 Problem 2: Lineage Outsourced to Fragile External Links

A more precarious situation arises when provenance information is not even located within the Hugging Face platform, but is instead hyperlinked to an external, third-party source.

Case Study 3: pfnet/plamo-2.1-2b-cpt

The model's homepage lacks a model tree, and its model card merely states that the model was derived by reducing the parameter size from an 8B-parameter model, without explicitly specifying the base model used for this operation. However, the model card provides links to three technical blogs for reference. By following these external links, we can infer that the mentioned 8B-parameter model is plano-2-8b.

Case Study 4: Remilistrasza/Mysterious_SDXL_Lah

Consider the Remilistrasza/Mysterious_SDXL_Lah model. Its Model Card is starkly empty of descriptive content, offering only a single hyperlink to a page on [Civitai](#), another model-sharing platform. To identify the base model, a user is forced to leave the Hugging Face ecosystem. Only upon navigating to this external page does one discover that the base model is “SDXL 1.0”.

This practice introduces two severe risks: **information fragmentation** and **link rot**. The integrity and availability of the lineage information become wholly dependent on the uptime and archival policies of an external website. Should the link become invalid (a 404 error), or the content on the remote page be altered or deleted, the model's traceable lineage is permanently severed. This practice renders the model's provenance fragile and unverifiable from within the Hugging Face ecosystem.

These documented failures, which range from information concealed in prose to its complete omission, demonstrate a clear and pressing need for an automated and reliable method to construct

model genealogies. The current reliance on inconsistent and manual documentation is an unsustainable bottleneck in an ecosystem that is scaling rapidly. To bridge this critical gap, we propose an approach to automatically identify and verify the derivative relationships between models, thereby enabling robust and scalable lineage tracing.

2 Benchmark

2.1 Construction

The construction of our benchmark involves a systematic pipeline that traces model derivations from base models to their descendants on Hugging Face. This process is divided into five key steps:

Step 1: Obtaining Base Model List. We begin by collecting a list of popular base models for the text-generation task from Hugging Face. Utilizing the HfApi provided by huggingface_hub, we retrieve the top 50 non-gated models sorted by the number of likes. The model identifiers are stored in a file named `base_models.txt`, where each line corresponds to one base model ID. This step is implemented via **Script 1**.

Step 2: Building the Model Derivation Tree. Based on `base_models.txt`, we recursively crawl derivative models to construct a “model forest”. For each base model, we explore its derivatives up to four levels deep, covering four derivation types: finetune, adapter, quantized, and merge. For each derivation type, a maximum of six sibling nodes are considered. Every node in the tree records metadata such as model name, crawl timestamp, depth level, and its child nodes. The resulting forest structure is serialized into `model_forest.json`. This step is implemented via **Script 2**.

Step 3: Constructing Model Pairs. From the model forest, we extract two categories of model pairs: adjacent parent-child pairs and non-adjacent ancestor-descendant pairs. By performing a Breadth-First Search (BFS) traversal on each tree, we generate:

- **Adjacent Pairs:** Direct parent-child model pairs.
- **Non-Adjacent Pairs:** All ancestor-descendant pairs where the ancestor is not the immediate parent.

These pairs are recorded in `adjacent_pairs.jsonl` and `non_adjacent_pairs.jsonl`, respectively. This step is implemented via **Script 3**.

Step 4: Single-Threaded Model Validation. To ensure data quality, we validate the structural correctness of all adjacent model pairs. For each model, we inspect its file tree on Hugging Face, checking for the presence of critical files such as `config.json`, `adapter_config.json`, and various parameter files (e.g., `.bin`, `.pt`, `.safetensors`, etc.). We verify whether the claimed derivation type aligns with the actual file structure and whether the parent model is valid. The results are compiled into a comprehensive validation report named `model_validation_report.json`. This step is implemented via **Script 4**.

Step 5: Filtering Erroneous Model Pairs. Based on the validation report, we remove all erroneous model pairs from the dataset. Models flagged with any inconsistencies or missing critical files are excluded. The cleaned set of adjacent model pairs is saved in

Table 1: Statistics of the GRANDMoTHER benchmark.

Statistic	Value
#Graphs	31
#Model pairs (edges)	248
Min / Median / Max pairs per graph	1 / 6 / 22
Avg. pairs per graph	8.0
#Models (nodes)	279
#Root models	31
#Intermediate models	44
#Leaf models	204
Share of leaf models	73.1%
Min / Avg / Max models per graph	2 / 9.0 / 23
Avg. out-degree of non-leaf models	3.31
Max fine-tuning depth	2

`adjacent_pairs_without_error.json`. This step is implemented via **Script 5**.

2.2 Characteristics

Our benchmark exhibits several notable characteristics that reflect its diversity and representativeness of the Hugging Face model ecosystem:

- **Variety of Derivation Types:** The dataset covers four major derivation types—finetune, adapter, quantized, and merge—capturing a wide range of model transformation patterns.
- **Hierarchical Depth:** With a recursive crawling depth of up to four levels, the benchmark contains both shallow and deep derivation chains, enabling analysis of both immediate and long-range model inheritance.
- **Scale and Coverage:** Starting from 50 popular base models, the recursive traversal results in a substantial model forest, ensuring coverage of highly influential models as well as their derivative variants.
- **Data Cleanliness:** Through rigorous validation and error filtering, the dataset ensures high structural integrity, making it suitable for tasks that require reliable model lineage information.
- **Real-World Relevance:** The benchmark reflects actual user-driven derivation patterns on Hugging Face, providing insights into community practices such as model fine-tuning trends, adapter usage, and quantization adoption.

3 Empirical Results

We design experiments to explore the following research questions.

- **Measurement Evaluation (RQ1):** How do different measurement metrics affect the effectiveness of LLM lineage detection?
- **Layers in Transformer Block Evaluation (RQ2):** How do different model layers influence the effectiveness of LLM lineage detection?
- **Graph Algorithms Evaluation (RQ3):** How do different graph algorithms influence the effectiveness of LLM lineage detection?
- **Failure Analysis (RQ4):** Why do existing methods fail to detect lineage?

Table 3: Lineage Detection Accuracy (%) w.r.t. Different Graph Algorithms (RQ3). Except for MoTher, we use timestamp as the direction method for other graph algorithms.

Graph Algorithm	Accuracy (%)	
Baseline (MoTher)	-	-
+ MoTher-Timestamp	-	-
+ Hard-Constraint	-	-
+ Undirected-Mst	-	-
+ atlas_snake_fan	-	-

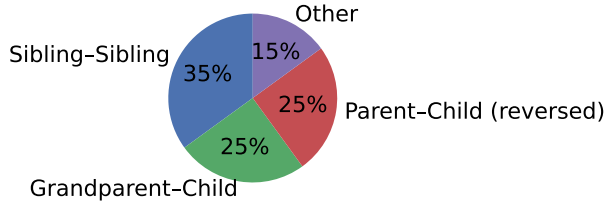
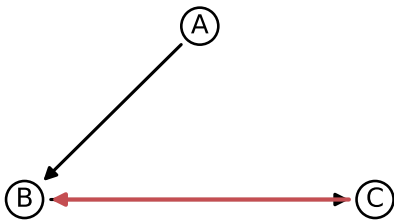


Figure 3: Proportion of different types of erroneous edges in the constructed lineage graph (RQ4).



Red arrow: erroneous sibling edge (C → B)

Figure 4: Case study illustrating erroneous edges in the constructed lineage graph (RQ4).

Table 2: Lineage Detection Accuracy (%) w.r.t. Different Measurement Metrics (RQ1)

Direction Method	L2	Cos.	Abs Mean	Abs Max	Sparse Abs.	Sparse Rel.
kurtosis	72.1	65.4	58.9	44.2	...	...
timestamp	72.1	65.4	58.9	44.2	...	...

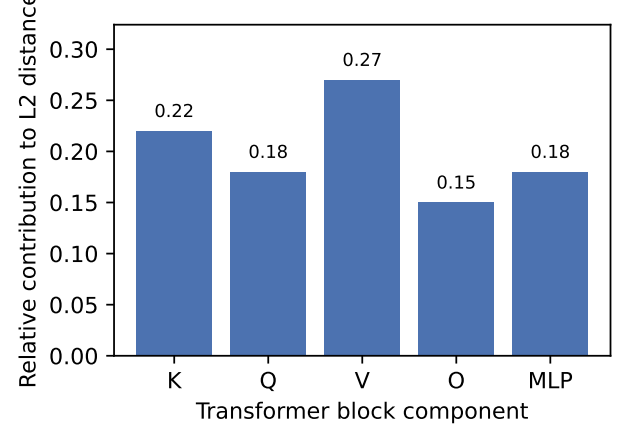


Figure 1: Contribution of different transformer block components (K/Q/V/O/MLP) to L2 distance (RQ2).

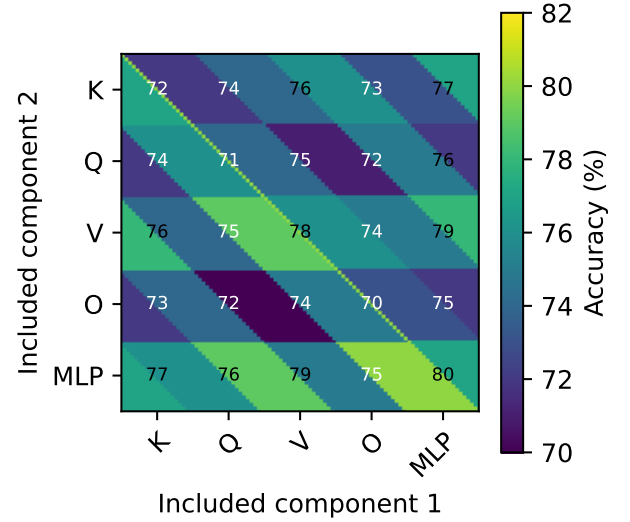


Figure 2: Accuracy of lineage detection with different combinations of transformer block components (RQ2).

3.1 RQ1

3.2 RQ2

3.3 RQ3

3.4 RQ4